MEAM 5200 Final Report

Group 6: Huayi Tang, Fanlian Zeng, Aditya Rangamani, Maanasa Rajeshwer

***Abstract*—This report covers the development, testing, and evaluation of a pick-and-place system for the MEAM 5200 final project, utilizing the Franka Emika Panda robot. The system is designed to autonomously handle static and dynamic blocks, focusing on maximizing score through stacking strategies and vision-based detection. Testing revealed challenges like simulation to hardware inconsistencies and robot-dependent alignment difficulties, providing insights into making the system more robust and efficient. This project highlights the importance of balancing precision, adaptability, and real-world limitations in robotics.**

***Index Terms*—Inverse Kinematics (IK), Forward Kinematics (FK), Motion Planning, Motion Prediction, Robotic Manipulation, Vision-based Detection, Robot Path Planning, Collision Avoidance, Real-time Control**

I. Introduction

The final project for the MEAM 5200 course focused on implementing a pick-and-place task using the Franka Emika Panda robotic manipulator. The challenge involved autonomously manipulating both static blocks arranged on a platform and dynamic blocks placed on a rotating turntable shared between two competing robots. The primary goal was to stack blocks on a designated platform, maximizing the competition score based on the block type (static or dynamic) and the height at which each block was placed within the stack.

The competition required designing a solution that balanced efficiency and precision while prioritizing safety to avoid disqualification due to collisions. To address this challenge, we implemented an Optimization-based Inverse Kinematics (IK) solver, leveraging known structural information of the environment and block positions provided in the global "World" frame. The project incorporated a vision system for real-time detection of block positions and orientations, enabling effective planning and execution of pick-and-place operations.

This report outlines our strategy for maximizing performance during the competition, including methods for static and dynamic block manipulation, collision avoidance, and scoring optimization. Section 2 details the scope of the competition strategy while Section 3 discusses the methods implemented and our competition strategies. Section 4 provides the testing and evaluation of our approach in both simulation and real-world scenarios. Section 5 provides a detailed analysis of these results. Section 6 describes the lessons we learned through this meaningful projects. These include conclusions and insights gained during lab testing and the final competition.

II. Scope

The primary objective of this project was to design and implement a robotic system capable of accurately detecting, grasping, and stacking static and dynamic blocks. The scope included:

- **Static Block Manipulation**: Ensuring precise detection and stacking of stationary blocks using predefined configurations and robust kinematic solutions.
- **Dynamic Block Manipulation**: Developing real-time tracking and motion prediction strategies to intercept and stack moving blocks from a rotating platform.
- **Collision Avoidance**: Implementing safety measures to prevent collisions with the stack, environment, or other obstacles.
- **Scoring Optimization**: Maximizing the competition score by prioritizing efficient stacking strategies and reliable block placement.

The project's scope encompassed iterative development and testing in both simulation and real-world hardware environments, ensuring robust performance under varying conditions.

III. Method

In this section, the general gaming strategies, including the static block grasping and stacking method, the dynamic block grasping and stacking method, scoring strategy in the limited time of the final competition, are proposed with their implementation details.

A. Scoring strategy

Based on the rules of the final competition, the strategy should maximize the scores in three minutes (5 minutes in knockout rounds). According to the scoring formula $Points = Value \times Altitude$ The more dynamic blocks you stack, the higher the altitude and the higher the score. Therefore, the key is to maximize the number of dynamic blocks grasped and stacked within a limited time. But, the reality is that grasping and stacking dynamic blocks is harder than static ones and will take more time. So, stacking 4 static blocks first is essential to ensuring a minimum score. All of our strategies only start to pick dynamic blocks after stacking 4 static blocks successfully. Based on these premises, we propose two strategies.

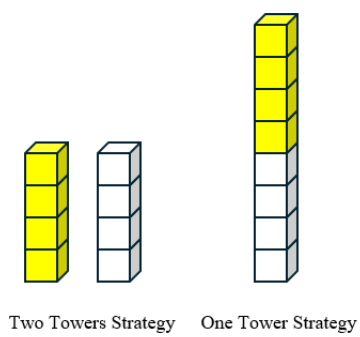

Fig. 1. Scoring Strategies

a) One Tower Stacking Strategy: Advantages of "One Tower Stacking Strategy" are that it takes less time and scores higher due to the high altitude of the dynamic blocks, and this strategy is easy to implement. Meanwhile, it has some drawbacks. Because grabbing dynamic blocks cannot guarantee that the block is in the middle of the gripper, it will cause the tower to become more distorted when stacking, so the higher the altitude, the easier it is to fall. Secondly, if the altitude of the stack is too high, it may exceed the joint limits, and the stacking posture should be carried out with a special configuration.

b) Two Towers Stacking Strategy: The advantage of this strategy is that because two relatively short towers are stacked, the difficulty is lower, the risk of falling is small, and if there is enough time, the dynamic block can be stacked on the tower of the static block, which can ensure that the pile is not distorted and not easy to fall. However, in practical implement, there are many problems, first of all, the orientation of the claws when stacking needs to be parallel to the tower to ensure that the stacking process will not knock down the other pile, and second, the score will be lower.

After testing the two strategies in the real robot, considering the problem of maximizing the score within the game time, we finally chose the "One Tower Stacking Strategy".

B. Path Planning Algorithm Selection

In lab 4, we tested several robot path planning algorithms such as RRT [3], [4] and Potential field [1] method. Based on our personal experience, these algorithms are very time-consuming to run, and they often fail to solve or hit obstacles during the planning process. So, we decided to use predefined configurations in every stage of the robot's actions. Fig 2 shows the procedures for determining these predefined configurations. Thanks to the simulation environment, we can find desired robot postures such as static block grasping and stacking posture, dynamic block grasping and stacking posture, as well as some middle-stage postures when actuating grasping or stacking actions. Using predefined configurations, the robot is faster and more stable [5] during grasping and stacking, and can also avoid all obstacles efficiently.

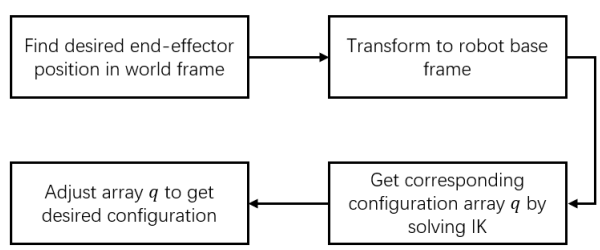

Fig. 2. Determination of Predefined Configuration

C. General Grasping Method

The key to grasping blocks is to ensure that the end-effector coordinate system coincides with the block's coordinate system by rotating the minimum angle [2] and then grabbing vertically from above. Since the Z axis of the block frame we get from the

camera is not always vertical downward (Direction of the end-effector's Z axis), we need to rotate the block frame to make their Z axis align; then it will be easier to calculate the rotation angle between these two frames. Fig. 3 shows how to align the Z-axis of the end effector frame with the block frame.

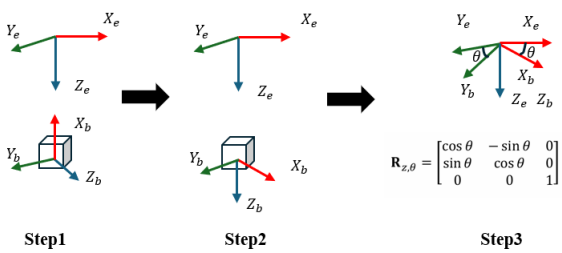

Fig. 3. Procures of Calculating the Rotation Angle of End-effector

Step 1: Obtain the block frame from the camera.
Step 2: Extract the rotation matrix from a homogeneous transformation matrix (see Fig. 4), and based on the rotation matrix determine which axis of the block frame aligns with the z-axis of the end effector frame.
Step 3: Rearrange columns of the rotation matrix, make it into stand form of the special rotation matrix that rotates only along the Z axis (see Fig. 3).
Step 4: Determine whether the z-axis of the block is aligned in the same direction as or opposite to the z-axis of the gripper. If they are opposite, use the rotation matrix to align their directions.
Step 5: Based on the expression of the Z-axis rotation matrix, solving the angle θ from corresponding $cos(\theta)$, and validate its sign by $sin(\theta)$
Step 6: Turn the angle into an acute angle and output the rotation matrix.

$$\mathbf{R}_1^0 = \begin{bmatrix} \hat{x}_1^0 \cdot \hat{x}_0 & \hat{y}_1^0 \cdot \hat{x}_0 & \hat{z}_1^0 \cdot \hat{x}_0 \\ \hat{x}_1^0 \cdot \hat{y}_0 & \hat{y}_1^0 \cdot \hat{y}_0 & \hat{z}_1^0 \cdot \hat{y}_0 \\ \hat{x}_1^0 \cdot z_0 & \hat{y}_1^0 \cdot \hat{z}_0 & \hat{z}_1^0 \cdot \hat{z}_0 \end{bmatrix}$$

Fig. 4. Rotation Matrix Example

Algorithm 1 Pose Adjustment Algorithm

Input: $pose_block$, H_ee_camera, $T_obj_to_end$
Output: Homogeneous Transformation matrix T
if $|pose_block[2,1]| > 0.93$ **then**
  Rearrange columns of $pose_pred[:3,:3]$ as:
  $pose_pred[:3,:3]$
  $= \text{Concatenate}\big(pose_pred[:3,2],$
  $pose_pred[:3,0], pose_pred[:3,1]\big)$
end if
if $|pose_block[2,0]| > 0.93$ **then**
  Rearrange columns of $pose_pred[:3,:3]$ as:
  $pose_pred[:3,:3]$
  $= \text{Concatenate}\big(pose_pred[:3,1],$
  $pose_pred[:3,2], pose_pred[:3,0]\big)$
end if
Compute $target_pose = H_ee_camera \cdot pose_pred$
if $target_pose[2,2] < 0$ **then**
  $target_pose = target_pose \cdot T_obj_to_end$
end if
Deep copy $target_pose$ to a
Find indices (x, y) of maximum absolute value in $a[:3,:3]$
Remove row x and column y from a
Compute $\theta = \arccos(a[0][0])$
if $|\sin(\theta) - a[1][0]| > 1e-3$ **then**
  $\theta = -\theta$
end if
if $\theta > \pi/2$ **then**
  $\theta = \theta - \pi/2$
else if $\theta < -\pi/2$ **then**
  $\theta = \theta + \pi/2$
end if
if $\theta > \pi/4$ **then**
  $\theta = \theta - \pi/2$
else if $\theta < -\pi/4$ **then**
  $\theta = \theta + \pi/2$
end if
Compute transformation matrix T as:

$$T = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

return T

D. Grab success detection

Grasping is not always successful due to various factors, especially for dynamic ones. In this case, it is important to determine whether the block was successfully grasped. Without this judgment, the robot arm will continue to stack the block, which may waste time and cause incorrect stack altitude. Our strategy is using arm.get_gripper_state() function to get the state feedback of the gripper. Then use sum(gripper_state['position']) to compute the distance between 2 gripper fingers, if this distance is larger than the threshold, grabbing is considered a success, and vice versa is considered a failure. The addition of this algorithm can reduce the cost of grasping failure.

E. Dynamic Block Motion Prediction

Pose prediction plays a crucial role in the successful grasping of dynamic blocks. Since the turntable rotates at a constant angular velocity and the distance of the block from the center of the turntable remains constant, the position and orientation of the block can theoretically be predicted with precision. Knowing the rotational speed of the turntable and the elapsed time during the prediction process, it is possible to estimate the prediction angle. This angle can then compute the homogeneous transformation matrix for the Z-axis (refer to Fig. 3), allowing us to derive the predicted pose of the detected dynamic block. An overview of this prediction strategy is illustrated in Fig. 5. **Algorithm 2** shows the whole prediction process:

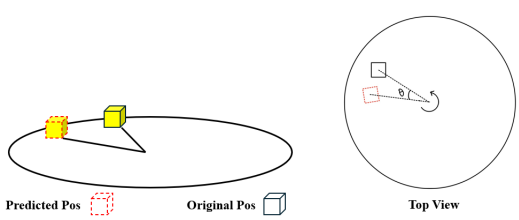

Fig. 5. Dynamic Block Pose Prediction

(Some descriptions of the symbol in superscript and subscript: c: camera frame, w: world frame, e: end-effector frame, b: robot base frame)

Step 1: The camera detects dynamic blocks and selects the block based on whether its x coordinate is within a certain range.

Step 2: Convert the pose of the detected block to the world frame.

Step 3: Set the prediction angle θ, then get the homogeneous transformation matrix base on the special rotation matrix along z-axis $H_{pre}(\theta)$(See Fig 3)

Step 4: Get the predicted pose of the detected block by pre-multiply the homogeneous transformation matrix $H_{pre}(\theta)$

Step 5: Calculate the target pose by converting the predicted pose to the end-effector frame

Algorithm 2 Pick Pose Dynamic

$H_c^e = detector.get_H_ee_camera()$
while True **do**
  detect all blocks in the camera's field of view
  $select \leftarrow$ None
  if the camera successfully detected the object **then**
    if Its x coordinate is in range(0.04, 0.1) **then**
      Select this block as to be grabbed
    end if
  end if
  if len$(list_dynamic_blocks) > 0$ **and** $select \neq$ None **then**
    $P_{block}^w = H_c^w \times P_{block}^c$
    Set prediction angle: θ
    $P_{block,pre}^w = H_{pre}(\theta) \times P_{block}^w$
    $P_{block,pre}^c = H_w^c \times P_{block,pre}^w$
    Target Pose $= H_c^e \times P_{block,pre}^c$
  end if
end while
return Target Pose

F. Stacking Strategy

Our stacking strategy is based on Grab success detection. For more stability and speed, we use the simplest stacking method, adding an offset to the stack height after each successful grab (each one is a block higher than the previous height).

G. Real robot parameter debugging

The biggest challenge we met was the differences between the gazebo simulation and the real robot.

These differences are reflected in the following aspects:

a) Environment setting: Dimensions of the hardware platform are not the same as the simulation. For example, the position of black, red, blue platform, and the height of the turntable are all different with the dimension in simulation. So when you take the code that works well in the simulation and use it on a real robot, you're likely to have problems. It is critical to adjust these errors quickly within the limited lab sessions.

b) Camera: In gazebo simulation, we get the pose of the block by directly reading from the model state topic of it, which means that there will be no errors or noise when detecting. But in real scenario, the pose of the block is obtained by detecting the April Tag. In this way, there are two problems, first, the detected April Tag is not always the top one, sometimes it will detect the tag on the side, and second, the detected pose will have a constant bias.

c) Physical property: In real scenario, physical properties have a bigger impact on block and robot. So grabbing and stacking that works in simulation sometimes doesn't work on a real robot, and it can even cause the existing tower to fall. If the dynamic block grasping prediction is inaccurate, the gripper will hit the surface of the block, causing errors and make the robot go offline.

Our first solution to these problems is adding April tag to blocks in simulation, by editing the urdf file of the blocks, we successfully added April Tag to all sides of these blocks. It is shown in Fig 6. Then we modified the algorithm of camera detecting in simulation, making it closer to real robot code. To handle bias between real robot and simulation, we force the end effector to move a fixed offset on its x y z axis by multiplying with the homogeneous transformation matrix:

$$H_{\text{trans}} = \begin{bmatrix} 1 & 0 & 0 & x \\ 0 & 1 & 0 & y \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Through this method, we can quickly adjust the coordinate system of the end effector during field debugging to eliminate the bias caused by the camera detection error.

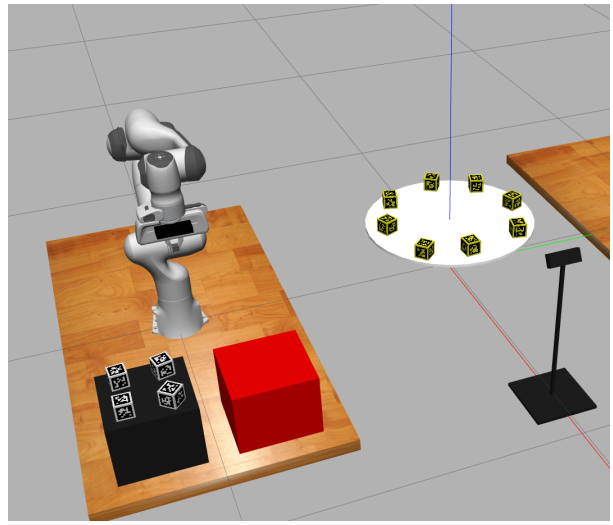

Fig. 6. April Tag Example in simulation

H. The whole process of the robotic block manipulation

The whole process of our strategy is shown in Fig 7, **Algorithm 3** shows more details of how to implement it.

IV. Evaluation

In this section, we made a comprehensive evaluation of the proposed method's performance, assessed through both simulation and hardware implementation.

A. Success Rate

The success rate is a critical metric for evaluating the effectiveness of a manipulation method. We tested the success rate of our method both on static blocks and on dynamic blocks in both simulation and real-world environments.

1) Success Rate for Static Manipulation: We first tested the success rate in picking and placing static blocks. We recorded a total of 5 times in our simulation to ensure accuracy. On the real robot, since time slots are limited and our algorithm only reached maturity during the final two lab sessions, we only recorded two groups of data. We also recorded the results in the final competition. Table I shows the success rate for static manipulation. As you can see, the success rate for static blocks is pretty high. We can always achieve a 100% success rate in the simulation environment. However, the

Algorithm 3 Robotic Block Manipulation for Blue Team

```
Initialize arm and object detector modules.
Move arm to starting position.
Wait for start signal.
if team is blue then
    Set gripper threshold and initialize FK and IK solvers.
    Initialize static and dynamic block lists.
    Define offsets and target positions for placing blocks.
    Static Block Handling:
    for each static block in the environment (4 iterations) do
        Obtain end-effector transformation using FK.
        Compute target pose using IK with pseudo-inverse method.
        Move arm to pick the block and close the gripper.
        Move arm back to starting position.
        if block successfully gripped (check gripper state) then
            Place block at target position based on its index in offset list.
            Move arm back to starting position.
        else
            Increment offset index.
            Open gripper and retry.
        end if
    end for
    Dynamic Block Handling:
    for each dynamic block detected (8 iterations) do
        Move arm to detection position.
        Use object detector to predict block pose.
        Adjust target pose based on block orientation and predicted motion.
        Compute target pose using IK with pseudo-inverse method.
        Move arm to pick the block and close the gripper.
        Move arm back to detection position.
        if block successfully gripped (check gripper state) then
            Place block at target position based on its index in offset list.
            Move arm back to starting position.
        else
            Increment offset index.
            Open gripper and retry.
        end if
    end for
end if
```

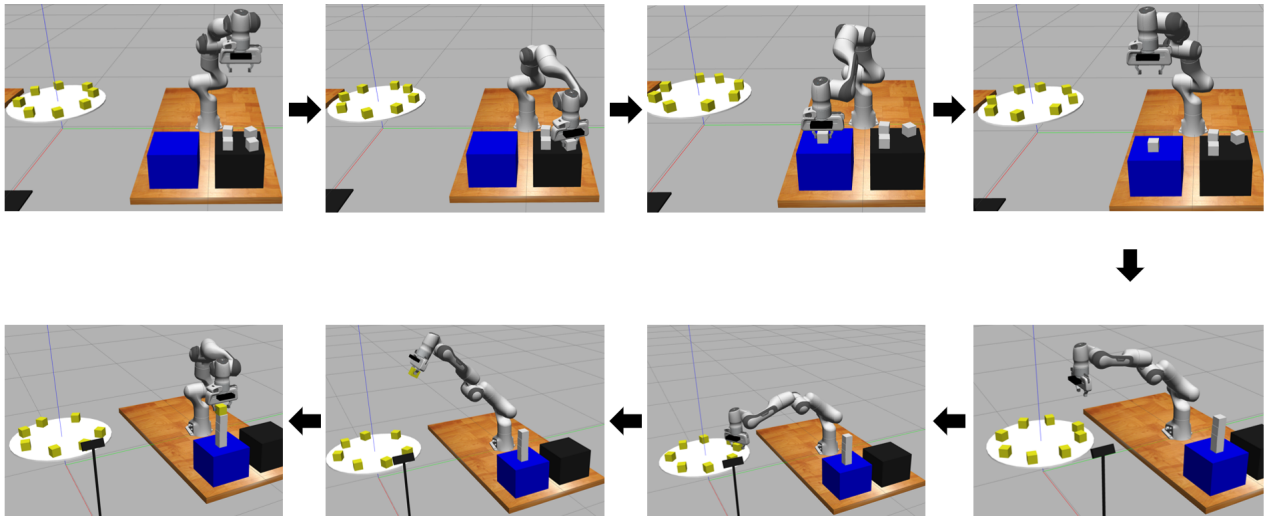

Fig. 7. The Whole Process

TABLE I
SUCCESS RATE FOR STATIC MANIPULATION

Environment	Number of Tests	Success Rates
Simulation	5	4/4, 4/4, 4/4, 4/4, 4/4 (Overall: 100%)
Real Robot	2	4/4, 3/4 (Overall: 88%)
Final Competition	2	3/4, 3/4 (Overall: 75%)

TABLE II
SUCCESS RATE FOR DYNAMIC MANIPULATION

Environment	Number of Tests	Success Rates
Simulation	5	4/4, 4/4, 4/4, 4/4, 4/4 (Overall: 100%)
Real Robot	2	4/4, 4/4 (Overall: 100%)
Final Competition	2	1/1, 4/4 (Overall: 100%)

success rate is lower on the real robot. We found that the reason is due to lighting conditions. The robot either cannot see four April tags at the same time or would recognize the side face of the block, leading to inaccurate information. We tried to change the position and the height of observing position, but it had little effect.

2) Success Rate for Dynamic Manipulation: We also recorded the success rate in picking and placing dynamic blocks. 5 groups of data is recorded in our simulation to ensure accuracy. On the real robot, only two groups of data are available due to the same reason as the static block. The results for the final competition are also reported. Table II shows the success rate for dynamic manipulation. As you can see, the success rate for dynamic blocks is extremely high. This is due to the accurate prediction of the dynamic block's motion. However, despite the high success rate in grasping, the gripper often fails to secure the block at its center, which poses future challenges of stacking that block. This phenomenon is more likely to occur in real-world environment. To address this limitation, we introduced a bias in the picking position to improve alignment and enhance stacking performance. This enables us to stack static blocks and dynamic blocks together. Figure 8 shows picking and stacking 8 blocks (4 static and 4 dynamic) in the simulation environment.

B. Manipulation Time

Since the competition has a strict time requirement, the success rate is meaningless if our algorithm runs very slowly. So, apart from the success rate, we also aim to make our manipulation algorithm faster. We reduce all the unnecessary poses and keep only the most important ones. Here, in table III and IV, we report the time for manipulation on both the static blocks and the dynamic blocks. These times include all phases of the process, including observation, picking, and stacking, for both static and dynamic scenarios.

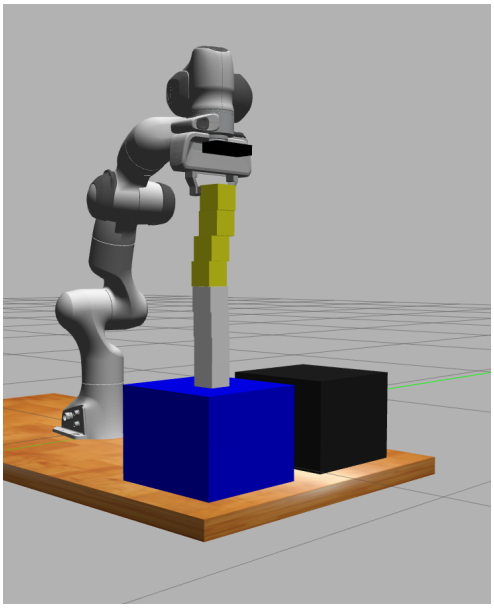

Fig. 8. Stacking 8 Blocks in Simulation Environment

TABLE III
MANIPULATION TIME FOR STATIC BLOCKS

Environment	Number of Tests	Manipulation Time (s)
Simulation	5	42.10, 49.16, 46.06, 47.53, 49.06 (Overall: 46.78)
Real Robot	2	27, 29 (Overall: 28)
Final Competition	2	25, 27 (Overall: 26)

As you can see from the table, the manipulation time in a real-world environment is always shorter than in a simulation. We assume that it is due to the fact that closing and opening the gripper takes a lot of time in the simulation. Additionally, the time for picking and stacking the dynamic blocks is almost the same as picking and stacking the static blocks. This shows the effectiveness and robustness of our dynamic motion prediction algorithm.

TABLE IV
MANIPULATION TIME FOR DYNAMIC BLOCKS

Environment	Number of Tests	Manipulation Time (s)
Simulation	5	46.66, 56.85, 44.62, 40.24, 50.50 (Overall: 47.77)
Real Robot	2	23, 25 (Overall: 24)
Final Competition	2	23, 24 (Overall: 23.5)

C. Limitation

Despite the relatively high success rate, the real world manipulator still suffers from various issues due to the uncertainty of the environment. We dig into each of them here and discuss potential solutions that could solve these problems.

1) April tag Issue: In Gazebo, ground truth is returned if we called the detection function of the block, which is really accurate. However, in the real world environment, April tag detection is used. One issue we encountered is that multiple detections may be returned if the camera sees both the front face and the side face of the block. It is hard to judge whether or not one is the front face or the side face since there is no way we can get information other than the name of the block and the pose of the block. We tried to mitigate this issue by making the observing position of the robot higher. However, it could not completely solve the problem. Figure 2 shows the situation in which the robot sees wrong April tag detections.

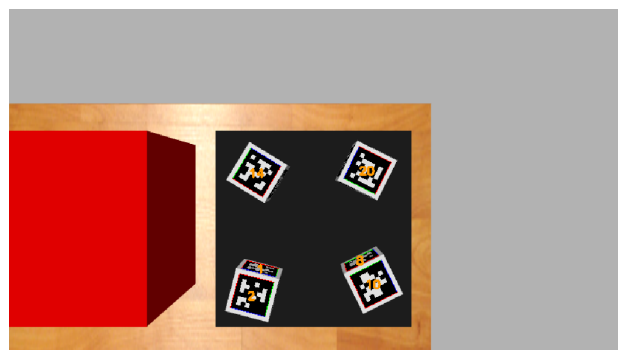

Fig. 9. The manipulator sees the wrong April tag detections

2) Stacking Height Issue: Another issue we encountered is that we cannot determine the current height of the blocks. For instance, if we attempt to stack a block and it falls off, the remaining stack will have a persistent offset because the robot is unaware of the fallen block. One potential solution is to detect the blocks that the robot is stacking onto. However, this approach introduces additional challenges, such as increased time consumption and difficulty in detecting the target blocks due to occlusion caused by the block currently being manipulated.

3) Dimension Issue: Another challenge we faced was the discrepancy between the dimensions in

real-world settings and those in simulation, especially the camera parameters. This mismatch caused the predicted dynamic block position to appear slightly ahead of the actual position for the red robot and slightly behind for the blue robot, resulting in inconsistent gripper alignment. To resolve this, we implemented a manual offset adjustment to the predicted positions, ensuring the gripper accurately targets the center of the block for reliable handling. However, the problem with this is that whenever the camera parameters are adjusted, we need to calibrate these manual offsets again.

D. Final Competition

In the final competition, we successfully picked up three static blocks and three dynamic blocks. However, due to limited time during the rehearsal session, we were unable to fine-tune the manual offset for grabbing dynamic blocks. This caused the gripping position to be slightly misaligned, falling behind the center of the block, which made it challenging to stack the blocks accurately. We believe that minor adjustments to the manual offset would enable successful stacking of all three static and three dynamic blocks. You can watch the full competition video at: https://youtu.be/Uf8M6tryFAY.

V. Analysis

In this section, we will analyze the results presented in the evaluation section comprehensively. We focused on the performance, advantage, and limitations of our algorithm as well as the analysis of the overall gaming strategy.

A. Analysis of Static Block Manipulation

According to the results in Section IV, you can see that the manipulation method of static blocks is quite robust in the simulation, achieving fast speed and very good accuracy at the same time. However, when it comes to the real-world environment, changing lighting conditions can make the detection noisy, which could potentially lead to a lower accuracy. We encountered the following three possibilities in real-world environment:

1) Seeing only the top face of the block. This situation is pretty good.
2) Seeing the top face and the side face of the block together. We tried to add a filter mechanism here to preserve only the top face of the block. But it is hard to distinguish which one is the top face and which one is the side face.
3) Seeing only the side face of the block. This situation is pretty bad.

The following two cases are the reasons for having a 75% success rate in real-world environment.

Though we did not optimize the moving path to be the shortest, our static block manipulation algorithm is really quick, considering the fewer-than-30-seconds runtime for each block on hardware. This validates the robustness of our coordinate alignment system since each time it can accurately and rapidly predict how to align the end effector with the block. This also shows that using predefined configurations is faster and more stable. We do not have to spend another 5-10 seconds waiting for planners like RRT [3], [4] or potential field [1] to return a feasible path. This ensures that we can complete the static block manipulation task within a 3-minute limit.

B. Analysis of Dynamic Block Manipulation

According to the results in Section IV, you can see that the manipulation method of dynamic blocks is very robust in both simulation and real-world environments. Our dynamic block manipulation can even be completed faster than the static blocks. This is mainly due to the robustness of angle alignment and dynamic motion prediction algorithm. From the experiment, we found that the angle alignment is more important on dynamic blocks than on static blocks. If the orientation of the gripper is wrong, either the gripper will hit the top face of the block and the algorithm will stop working or the gripper will miss the block. Thus, the success rate of the dynamic blocks shows the robustness of our algorithm. Additionally, the motion prediction algorithm is also very accurate. Since we only pick blocks that are within a predefined range, a fixed prediction angle is good enough under this circumstance. This was further validated by our final competition results since we can pick up and stack a dynamic block within 30 seconds.

However, using a motion prediction strategy implies that we rely heavily on the camera parameters. Changing the camera parameters will result in an offset between the predicted position and the actual

position. This happens when we perform sim-to-real transfer. We had to manually add an offset along the x-axis of the end effector frame in order to tackle this problem. During the final competition, since the rehearsal time is limited, we are unable to find the new offset in such a limited period. This led to a consistent positional lag in our predictions during the competition.

Another potential issue is the use of a fixed prediction angle. If our opponent accidentally slows down or stops the turntable, the prediction algorithm could fail because it assumes the turntable is still rotating at a constant speed. However, such cases are extremely rare, and the current performance of our dynamic manipulation system remains highly effective.

C. Competition Strategy Analysis

Our strategy was largely successful in the experiments and the final competition. However, instead of stacking the dynamic block directly, putting the dynamic block on the black center first and treating it as another static block could be a better idea. This adjustment would help mitigate issues caused by misalignment with the block's center and reduce the impact of changes to the camera matrix.

VI. Lessons Learned

A. Sim to Real Transfer is Complex and Time Consuming

The sim-to-real transfer is a complex yet essential challenge in robotics, as demonstrated in pick-and-place final project. The differences between simulation and real-world environments introduced several difficulties, including discrepancies in dimensions, such as positions of the camera and positions and height of the turntable, lighting conditions, camera parameters matrix, and physical properties such as the angular velocity of the turntable. For instance, the simulation provided precise ground truth for block detection, while the real robot relied on April Tag detection, which was prone to errors due to lighting changes and multi-face detection issues. The offset in dynamic block manipulation further highlighted the sensitivity of prediction algorithms to real-world turntable speeds and camera parameters. Another difference is that when a real-world manipulator touches the top face of the block, the manipulator will throw out an error and the algorithm will stop running. However, in the simulation, we found out that the gripper can actually flip the block and make the block turn 90 degrees before it grabs the block. These kinds of behavior and parameter differences make testing on the real robot more necessary as long as the algorithm performs well in the simulation. Unfortunately, due to limited lab time slots, we are unable to test the algorithm completely in the real robot. This suggests that we should start real-world testing as soon as possible.

B. Accounting for Uncertainty is Important in Algorithm Development

Accounting for uncertainty, especially in the real-world scenario, is key to developing a successful algorithm. For instance, the detection of successful grabbing and stacking height issue tackling may seem unnecessary if the block to be placed is exactly on top of the center of the stack. However, there would be a lot of issues if these kinds of detection function are not implemented since there are lots of uncertainties. Additionally, hard coding everything is not a good choice since there may be some dimension differences between simulation and real-world. Recalculating the joint angles q manually could take unnecessary time, making the schedule in the lab session tighter. Instead of recalculating, we leveraged the predefined q from simulation as an initial guess, allowing the inverse kinematics solver to refine the solution dynamically. This approach ensures a rapid solution from IK while maintaining the flexibility to adapt to real-world uncertainties.

C. Review the Lecture if Something Went Wrong

We went through some obstacles when we tried to implement the dynamic motion prediction algorithm. After reviewing and thinking our approach comprehensively, we would always find that there would be some knowledge that we haven't completely understood during the lecture, such as the distinctions between pre-multiplying and post-multiplying rotation matrices and interpreting the signs within the homogeneous transformation matrix. Revisiting these lecture materials always resolved our issues and proved very useful in developing a robust dynamic block motion prediction

algorithm. The lectures not only provided theoretical insights but also served as a critical resource for practical problem-solving throughout our project.

VII. Conclusion

The MEAM 5200 final project provided a comprehensive challenge in developing a robust, efficient, and precise robotic pick-and-place system. Through the integration of forward and inverse kinematics, motion prediction algorithms, and real-time vision-based detection, we successfully demonstrated a capable solution for handling both static and dynamic blocks. Our implementation of predefined configurations, robust error handling mechanisms, and real-time adjustments addressed real-world challenges effectively, leading to notable performance in simulation and hardware testing.

Key lessons learned included the complexities of sim-to-real transfer, the importance of accounting for environmental uncertainties, and the value of iterative testing and debugging. While challenges like stacking alignment and environmental inconsistencies remain, this project lays a strong foundation for further advancements in developing autonomous robotic manipulation systems.

References

[1] Y. K. Hwang and N. Ahuja, "A potential field approach to path planning," in IEEE Transactions on Robotics and Automation, vol. 8, no. 1, pp. 23-32, Feb. 1992, doi: 10.1109/70.127236. keywords: Path planning;Orbital robotics;Robot kinematics;Heuristic algorithms;Shape;Electrostatics;Planing;Sun;Joining processes;Motion planning,

[2] P. E. Hart, N. J. Nilsson and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," in IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, July 1968, doi: 10.1109/TSSC.1968.300136. keywords: Costs;Mathematical programming;Minimization methods;Functional programming;Automatic control;Minimax techniques;Gradient methods;Chemical technology;Automatic programming;Instruction sets,

[3] S. Karaman, M. R. Walter, A. Perez, E. Frazzoli and S. Teller, "Anytime Motion Planning using the RRT*," 2011 IEEE International Conference on Robotics and Automation, Shanghai, China, 2011, pp. 1478-1483, doi: 10.1109/ICRA.2011.5980479. keywords: Trajectory;Planning;Vehicles;Mobile robots;Heuristic algorithms;Vehicle dynamics,

[4] S. Karaman, M. R. Walter, A. Perez, E. Frazzoli and S. Teller, "Anytime Motion Planning using the RRT*," 2011 IEEE International Conference on Robotics and Automation, Shanghai, China, 2011, pp. 1478-1483, doi: 10.1109/ICRA.2011.5980479. keywords: Trajectory;Planning;Vehicles;Mobile robots;Heuristic algorithms;Vehicle dynamics,

[5] W. M. Grimm, N. Becker and P. M. Frank, "Robust stability design framework for robot manipulator control," Proceedings. 1988 IEEE International Conference on Robotics and Automation, Philadelphia, PA, USA, 1988, pp. 1042-1047 vol.2, doi: 10.1109/ROBOT.1988.12198. keywords: Robust stability;Manipulators;Robot control;Robust control;Design methodology;Control systems;Stability analysis;Control nonlinearities;Force control;Nonlinear control systems,